Clase de Complexitate

2024

Cuprins

1 Motivație **1**

2 Obiective **1**

3 Clase de Complexitate **1**
3.1 Cele mai grele probleme din NP 4

4 Ierarhia Claselor de Complexități **5**
4.1 Rezumat ierarhie - P vs NP 7

5 Analiza Problemă - SAT **7**
5.1 $SAT \in NP$ 8
5.2 $SAT \in NP\text{-}Complete$ - Teorema Cook-Levin 8
5.3 $3SAT \in NP - Complete$ 10

1 Motivație

În cursul precedent ne-am propus să explorăm limitele calculabilității. Am descoperit astfel că există probleme pentru care nu putem implementa o soluție care să meargă pentru orice fel de intrare, indiferent de modelul de calcul ales. În continuare, ne propunem să clasificăm problemele care sunt calculabile în funcție de abilitatea noastră de a le rezolva eficient. Vom descoperi că există o categorie vastă de probleme, cu aplicații practice în foarte multe domenii, pentru care nu am descoperit soluții care să funcționeze *corect* și să consume o cantitate *rezonabilă* de resurse (ex: timp, spațiu) pentru *orice* intrare validă. Discuția din acest capitol se va concentra pe analiza dificultății problemelor de decizie.

2 Obiective

Ne propunem să:

- Identificăm principalele clase de complexitate și relațiile dintre acestea.
- Introducem mecanismul de reducere polinomială pentru a compara dificultatea rezolvării unor probleme de referință.
- Analizăm problema SAT, ca punct de pornire în descoperirea problemelor dificile pentru care nu cunoaștem rezolvări eficiente.

Observație 1. *Înainte să parcurgeți acest curs vă recomandăm să vă familiarizați cu noțiunile predate în cursul despre modele de calcul, în particular descrierea unei Mașini Turing (not. MT).*

3 Clase de Complexitate

Definim funcțiile de complexitate $time_M : \mathbb{N} \to \mathbb{N}$, respectiv $space_M : \mathbb{N} \to \mathbb{N}$, particularizate pentru o Mașină Turing M, care se oprește pentru orice intrare. Atunci:

Definiție 1. *Complexitatea temporală ($time_M(n)$) reprezintă numărul maxim de tranziții realizate de MT M pentru orice intrare de dimensiunea n.*

Definiție 2. *Complexitatea spațială ($space_M(n)$) reprezintă numărul maxim de celule scanate de M pentru orice intrare de dimensiunea n.*[1]

Ne propunem să grupăm problemele în funcție de complexitatea celei mai eficiente MT care le rezolvă.

Definiție 3. *$TIME(t(n))$ = mulțimea problemelor de decizie care admit o rezolvare cu complexitatea temporală $O(t(n))$.*

În particular, folosim notația $DTIME(t(n))$ pentru a ne referi la rezolvări care folosesc o MT **deterministă**, respectiv $NTIME(t(n))$ pentru a ne referi la rezolvări care apelează la o MT **nedeterministă**.

Definiție 4. *$SPACE(s(n))$ = mulțimea problemelor de decizie care admite o rezolvare cu complexitatea spațială $O(s(n))$.*

În particular, folosim notația $DSPACE(s(n))$ pentru a ne referi la rezolvări care folosesc o MT **deterministă**, respectiv $NSPACE(s(n))$ pentru a ne referi la rezolvări care apelează o MT **nedeterministă**.

Definiție 5: Clasa PTIME (P)

Mulțimea problemelor de decizie care pot fi rezolvate de un algoritm *determinist* **polinomial**.

$$P = \bigcup_k DTIME(n^k)$$

Exemplu 1: Probleme din clasa P

- Există un element X într-o listă L?
- Există un drum elementar de la nodul A la nodul B într-un graf orientat?
- Este un graf bipartit?

Definiție 6: Clasa NPTIME (NP)

Mulțimea problemelor de decizie care pot fi rezolvate de un algoritm *nedeterminist* **polinomial**.

$$NP = \bigcup_k NTIME(n^k)$$

Fie w datele de intrare ale unei probleme de decizie Q. Atunci c este un **certificat** al problemei de decizie Q dacă există un algoritm verificator care, având acces la w și informația adițională din c poate valida că $Q(w) = True$.

Definiție 7: Clasa NPTIME (NP)

(Alternativ) Mulțimea problemelor de decizie care admit existența unui certificat care poate fi verificat în timp *determinist* **polinomial**[a].

[a] Implicit, mărimea certificatului necesar c trebuie să fie polinomială în raport cu dimensiunea intrării ($|c| \leq p(n)$, unde $p()$ este un polinom).

[1] În practică, atunci când ne referim la complexitatea spațială asimptotică a unui algoritm, ne propunem să măsurăm doar memoria suplimentară celei necesare pentru a reprezenta datele de intrare. În acest mod putem diferenția între algoritmi care folosesc spațiu adițional subliniar (e.g. $O(log(n))$ vs. $O(\sqrt{n})$). Pentru a explora acest concept putem folosi ca model de calcul o MT cu o banda pentru datele de intrare 'read-only', care nu contribuie la complexitatea spațială, respectiv una sau mai multe benzi ale căror celule pot fi modificate.

Exemplu 2: Probleme din clasa NP

- Există un drum elementar cu cel *puțin* K noduri intermediare de la A la B?
- Putem planifica n procese, cu constrângeri de timp (deadline, durată), pe m mașini?
- Putem atribui o culoare c ($1 \leq c \leq K$) fiecărui nod dintr-un graf, astfel încât fiecare nod să fie colorat diferit față de vecinii săi?

Observație 2. *Dacă P descrie clasa problemelor de decizie pentru care putem* ***construi eficient***[2] *un certificat, în clasa NP sunt grupate acele probleme pentru care putem doar să* ***verificăm eficient*** *dacă un certificat de dimensiune polinomială demonstrează că problema are soluția $True$.*

Definiție 8: Clasa EXPTIME (EXP)

$$EXPTIME = \bigcup_k DTIME(2^{n^k})$$

Exemplu 3: Probleme din clasa EXP

- Se oprește o MTD în K pași pe o intrare w?
- Verificarea dacă o mutare garantează victoria într-un joc generalizat de șah/Go, pe o tablă cu $n \times n$ celule.

Definiție 9: Clasa PSPACE

Mulțimea problemelor de decizie care pot fi rezolvate de un algoritm *determinist* cu o complexiatea spațială **polinomială**.

$$PSPACE = \bigcup_k DSPACE(n^k)$$

Exemplu 4: Probleme din clasa PSPACE

- Descriu două expresii regulate același limbaj?
- Fie un sistem compus din mai multe procese care pot comunica între ele. Dată fiind descrierea unei stări inițiale este posibilă atingerea unei stări de deadlock?
- Verificarea dacă o mutare garantează victoria într-un joc generalizat de Reversi (tablă de $n \times n$ celule), Super Mario Bros, etc.

Definiție 10: Clasa NPSPACE

Mulțimea problemelor de decizie care pot fi rezolvate de un algoritm *nedeterminist* cu o complexiatea spațială **polinomială**.

$$NPSPACE = \bigcup_k NSPACE(n^k)$$

[2] În acest context, folosim termenul de eficient ca sinonim pentru timp polinomial determinist.

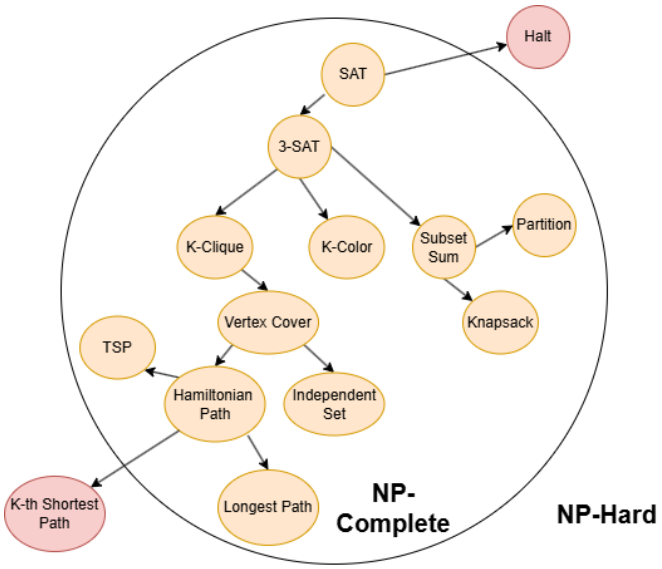


Figura 1: O parte din problemele NP-Complete, respectiv NP-Hard explorate la curs.

3.1 Cele mai grele probleme din NP

Pentru a descoperi *cele mai grele* probleme din clasa NP introducem mecanismul de **reducere**.

Definiție 11: Reducere polinomială (many-one)

Fie A și B, două probleme de decizie. Fie o funcție $F : Input_A \to Input_B$, definită pe intrările corespunzătoare problemei A, care produce intrări valide pentru problema B. Prespunem că F poate fi calculată de un algoritm determinist cu complexitate temporală polinomială. Atunci, spunem că A se reduce polionomial la B ($A <_p B$) dacă:

$$A(w) = 1 \iff B(F(w)) = 1$$

Dacă $A <_p B$, atunci putem rezolva A, apelând direct algoritmul corespunzător lui B pe intrarea modificată. Astfel, putem să facem câteva afirmații referitoare la dificultatea relativă a rezolvării celor două probleme:

- Complexitatea rezolvării problemei B ne oferă o limită *superioară* pentru oricare rezolvare directă a problemei A.
- Complexitatea rezolvării problemei A ne oferă o limită *inferioară* pentru oricare rezolvare directă a problemei B.

Lema 1. *Relația $<_p$ este tranzitivă. Astfel, $A <_p B$ și $B <_p C$ implică $A <_p C$.*

Definiție 12: NP-Hard

A $\in$ NP-Hard $\iff \forall Q \in NP \;.\; Q <_p A$

În NP-Hard se află toate problemele *cel puțin* la fel de grele ca toate problemele din NP. În NP-Hard se află însă și probleme mai grele decât cele din NP (de exemplu, Problema Opririi).

Definiție 13: NP-Complete

A $\in$ NP-Complete $\iff A \in$ NP-Hard AND $A \in NP$.

Soluția unei probleme NP-Complete (NPC) poate fi folosită pentru a rezolva **orice** problemă din NP. Din acest motiv considerăm, informal, că problemele NPC sunt *cele mai grele* probleme din NP.

4 Ierarhia Claselor de Complexități

Fie o problemă de decizie calculabilă *Prob*.

Lema 2. *$Prob \in DTIME(t(n)) \rightarrow Prob \in DSPACE(s(n))$. De asemenea, $Prob \in NTIME(t(n)) \rightarrow Prob \in NSPACE(s(n))$.*

Demonstrație. O mașină Turing a cărei execuție pe o intrare w se termină în $t(n)$ pași nu poate accesa mai mult de $t(n)$ celule. □

Corolar 1. $Prob \in P \rightarrow Prob \in PSPACE$

Corolar 2. $Prob \in NP \rightarrow Prob \in NPSPACE$

Lema 3. $Prob \in DTIME(t(n)) \rightarrow Prob \in NTIME(t(n))$

Demonstrație. Un algoritm determinist poate fi simulat direct pe un model de calcul nedeterminist. Prin urmare, dacă problema admite o rezolvare în $O(t(n))$ pe un model de calcul determinist, admite și o rezolvare cu aceeași complexitate pe un model de calcul nedeterminist. □

Corolar 3. $Prob \in P \rightarrow Prob \in NP$

Lema 4. *Fie $f(n) \geq n \ \ \forall n \in \mathbb{N}$. Atunci $Prob \in SPACE(f(n)) \rightarrow Prob \in TIME(2^{O(f(n))})$.*

Demonstrație. Care este numărul maxim posibil de configurații prin care poate trece o mașina Turing care accesează f(n) celule fără a cicla? O configurație este identificată prin trei parametri:

- Q (starea MT)
- B (poziția capului de bandă)
- T (valorile scrise efectiv pe cele f(n) celule accesibile)

Observăm că:

- Numărul de stări este independent de datele de intrare.
- $1 \leq B \leq f(n)$.
- Avem $|\Sigma|^{f(n)}$ posibilități distincte de a completa banda.

Prin urmare, numărul total de configurații prin care poate trece MT este $|Q| \cdot f(n) \cdot |\Sigma|^{f(n)} \in 2^{O(f(n))}$. □

Corolar 4. $Prob \in PSPACE \rightarrow Prob \in EXP$

Teoremă 1: Teorema lui Savitch

Fie $s(n) \geq n$. Atunci $Prob \in NSPACE(s(n)) \rightarrow Prob \in DSPACE(s(n)^2)$

Demonstrație. Fie graful orientat $G_{MT}(V, E)$ determinat de configurațiile unei mașini Turing nedeterministe M care accesează cel mult $s(n)$ celule. Conform Lemei 4, numărul maxim de configurații este $|V| = 2^{O(s(n))}$. Considerăm că avem muchie între două noduri A și B dacă este posibilă o tranziție între cele două configurații asociate nodurilor, conform specificației MT.

Walter Savitch a demonstrat că este posibil să identificăm eficient, din punct de vedere al spațiului, dacă există o cale de la configurația inițială la o configurație care corespunde unei stări de acceptare a intrării. Concret, Savitch a propus următorul algoritm pentru rezolvarea problemei PATH (”*Există un drum de la un nod sursă S la un nod destinație D într-un graf orientat $G(V, E)$?*”):

Algorithm 1 $KPATH$ verifică dacă există un drum de la S la D în $G(V, E)$ format din cel mult K pași intermediari.

```
1:  function KPATH(G(V,E), S, D, K)
2:      if K == 0 then
3:          return s == d
4:      end if
5:      if K == 1 then
6:          return (s, d) ∈ E
7:      end if
8:      for v in V do                                   ▷ Alegem un nod intermediar
9:          if KPATH(S, v, ⌊K/2⌋) and KPATH(v, D, ⌈K/2⌉) then
10:             return True                             ▷ Verificăm dacă acesta se poate afla la jumătatea distanței
11:         end if
12:     end for
13:     return False
14: end function
15: function PATH(G(V,E), S, D)
16:     return KPATH(G, S, D, |V| − 1)
17: end function
```

Algoritmul $KPATH$ realizează $O(log(|V|))$ apeluri recursive, fiecare apel reținând $O(log(|V|))$ informații pe stivă. Astfel, complexitatea spațială a algoritmului prezentat este $O(log^2(|V|))$.

Observăm că pentru a aplica algoritmul $KPATH$ pe graful configurațiilor G_{MT} **nu este necesar să îl reprezentăm explicit în memorie**. Avem nevoie doar de un mecanism pentru a itera prin mulțimea configurațiilor (linia 8) și a verifica dacă este posibilă tranziția dintre două configurații (linia 6). Conform ipotezei, pentru a reține o configurație avem nevoie de $O(s(n))$ memorie.

Prin urmare, putem folosi o MT **deterministă** pentru a verifica dacă MT nedeterministă M acceptă o intrare cu o complexitate spațială de:[3]

$$O(log^2(|V|)) = O(log^2(2^{O(s(n))})) = O(s(n)^2)$$

□

Corolar 5. $PSPACE = NPSPACE$

Corolar 6. $Prob \in NP \rightarrow Prob \in PSPACE$

Demonstrație. Afirmația reiese direct din Corolarul 2, respectiv 5. □

Teoremă 2

Fie $t : \mathbb{N} \rightarrow \mathbb{N}$, o funcție totală calculabilă cu o complexitate temporală de $O(t(n))$[a], $t(n) \geq nlog(n)$. Atunci există o problemă care poate fi rezolvată cu o complexitate temporală de $O(t(n))$ dar nu în $o(t(n)/log(t(n))$.

[a] Dată fiind o intrare formată din n biți de 1, există o MT care scrie pe bandă valoarea $t(n)$ în $O(t(n))$ pași.

Corolar 7. $\forall\, 1 \leq \alpha_1 \leq \alpha_2 \,.\, PTIME(n^{\alpha_1}) \subset PTIME(n^{\alpha_2})$

Corolar 8. $P \subset EXP$

Teoremă 3

Fie $s : \mathbb{N} \rightarrow \mathbb{N}$, o funcție totală calculabilă cu o complexitate spațială de $O(s(n))$, $s(n) \geq log(n)$. Atunci există o problemă care poate fi rezolvată cu o complexitate spațială de $O(s(n))$ dar nu în $o(s(n))$.

Observație 3. *Informal, ultimele două teoreme ne confirmă intuiția că, dacă am avea la dispoziție mai mult timp/spațiu am putea rezolva un număr strict mai mare de probleme decât dacă am avea mai puțin timp, respectiv spațiu.*

[3] O mică dificultate apare de la faptul că trebuie să știm valoarea lui $s(n)$ pentru a determina valoarea maximă a parametrului K. Putem depăși această dificultate apelând incremental KPATH pentru valori din ce în ce mai mari ale lui K $(n, n+1, n+2, \dots)$. Dacă pentru o anumită valoare nu este posibil să atingem, din configurația inițială, nicio configurație de lungime K, întoarcem $False$. Altfel, dacă descoperim o cale spre o configurație de acceptare, întoarcem $True$.

4.1 Rezumat ierarhie - P vs NP

Am descoperit următoarea ierarhie:

$$P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXP \tag{1}$$
$$P \subset EXP \tag{2}$$

Este $P = NP$? Din păcate, nu știm răspunsul la această întrebare, deși ne-am dori foarte mult să îl aflăm. Dacă cele două clase ar fi identice multe probleme cu aplicații practice importante ar admite, teoretic, o rezolvare eficientă. Pentru a demonstra această echivalență ar fi suficient să găsim o rezolvare eficientă pentru o singură problemă din $NP - Complete$. Cu toate acestea, faptul că, în ciuda eforturilor susținute de-a lungul ultimilor decenii, nu s-a descoperit o astfel de rezolvare sugerează că, cel mai probabil, cele două clase sunt distincte.

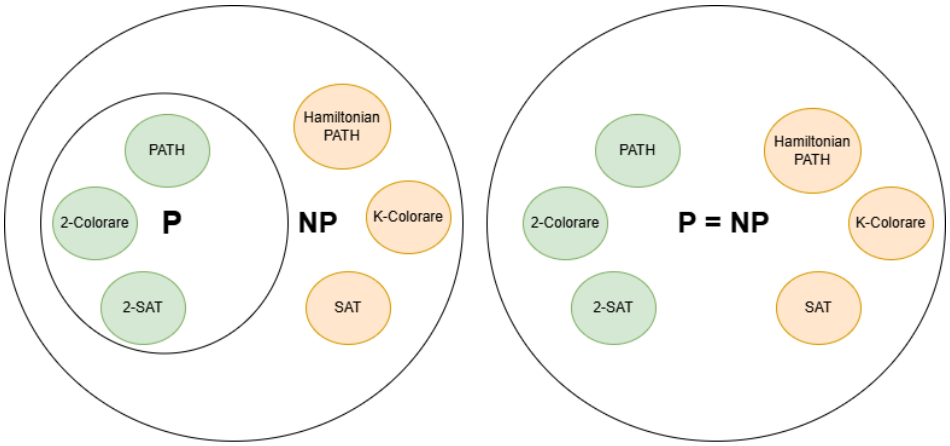


Figura 2: P vs. NP

5 Analiza Problemă - SAT

Problema 1: SAT

Date de intrare:

- O formulă logică booleană ϕ, scrisă în forma normal conjunctivă (CNF), cu n variabile și m clauze.

Problema de decizie:

- Există o atribuire de valori boolene (True/False) variabilelor care compun ϕ astfel încât formula să fie adevărată? (Este ϕ satisfiabilă?)

Exemplu:

$$(\overline{s} \vee a \vee b) \wedge (\overline{s} \vee \overline{a} \vee \overline{b}) \wedge (s \vee a \vee \overline{b}) \wedge (s \vee \overline{a} \vee b)$$

Explicație: a, b, s și c sunt variabilele folosite. Fiecare apariția a unei variabile într-o clauză, fie în forma a sau ceva negată $\overline{a}$, reprezintă un literal. Observăm că formula este în forma CNF deoarece este compusă dintr-o conjuncție de clauze, fiecare clauză fiind o disjuncție de literali.

Follow-up 1:

Cum putem rescrie în forma CNF formula $a \rightarrow b$?

Follow-up 2:

Dacă considerăm a și b cunoscute, ce valori trebuie să aibă s pentru ca formula de mai sus să fie satisfiabilă?

5.1 $SAT \in NP$

Demonstrație.

Plan demonstrație: Pentru a arăta că o problemă aparține NP putem propune un algoritm nedeterminist polinomial care obține răspunsul corect pentru intrare.
Uzual, o astfel de rezolvare are două etape:

1. Construim un certificat de dimensiune polinomială. Pentru a determina fiecare componentă a certificatului folosim operatorul *choice*(), care se evaluează în $O(1)$.
2. Verificăm determinist certificatul, în timp polinomial.

Concret, pentru problema SAT, un certificat ar fi o asignare de valori pentru fiecare variabilă. Pasul de verificare presupune verificarea că, pentru fiecare clauză, cel puțin un literal are asignată valoarea $True$.

Algorithm 2 Rezolvare nedeterministă a problemei SAT

```
1:  function SAT(Variables, Clauses)
2:      VarValues = {}
3:      for variable in Variables do          ▷ Alegem o valoare de adevăr pentru fiecare variabilă - O(n)
4:          VarValues[variable] = choice({True, False})
5:          VarValues[¬variable] = not VarValues[variable]
6:      end for
7:      for clause(x, y, z) in Clauses do     ▷ Verificăm că fiecare clauză este satisfiabilă - O(m)
8:          if !VarValues[x] and !VarValues[y] and !VarValues[y] then
9:              fail()
10:         end if
11:     end for
12:     success()
13: end function
```

Deoarece *choice*() se realizează în $O(1)$, complexitatea temporală a întregului algoritm este: $\theta(n) + \theta(m)$. Complexitatea spațială este $O(n)$, deoarece reținem cele n alegeri în dicționarul $varValues$.
Dacă ne-am propune să simulăm determinist însă acest algoritm, complexitatea sa ar fi $O(2^n)$ datorită pasului de generare a certificatului. Din păcate, pentru această problemă nu s-a descoperit încă o soluție determinist cu o complexitate temporală subexponențială.

□

5.2 $SAT \in NP\text{-}Complete$ - Teorema Cook-Levin

Demonstrație. O problemă este $NP-Complete$ dacă aparține atât mulțimii NP, cât și $NP-Hard$. Am demonstrat mai sus că $SAT \in NP$. Pentru a demonstra că $SAT \in NP - Hard$, vom arăta că orice problemă din clasa NP se reduce polinomial la SAT.

Fie $Prob$ o problemă din NP, respectiv w o intrare validă arbitrară ($|w| = n$). Fie NT o MT nedeterministă care rezolvă orice instanță a problemei $Prob$ în timp polinomial ($p(n)$).

Observație 4. *Deoarece execuția lui NT se termină în $p(n)$ pași, atunci numărul maxim de celule scanate pe bandă este tot $p(n)$ (v. Lema 2).*

Plan demonstrație: Ne propunem să codificăm, în timp polinomial, execuția lui NT pe intrarea w ca o formulă booleană ϕ, astfel încât $NT(w) = 1 \iff SAT(\phi) = 1$. Configurația MT la un pas de execuție i este compusă din:

- Q (starea în care se află MT)
- B (poziția capului de bandă)
- T (valorile scrise efectiv pe cele p(n) celule accesibile)

Notăm cu $S = |Q|$ numărul de stări, cu I id-ul stării inițiale, respectiv cu A id-ul stării de acceptare a intrării. Pentru a defini ϕ vom introduce următoarele variabile pentru a descrie complet configurațiile posibile din fiecare pas de execuție:

Nume variabilă	Semnificație indecși	Domeniu	Număr variabile
$Q_{i,j}$	$i \rightarrow$ pasul de execuție $j \rightarrow$ id-ul stării	$0 \leq i \leq p(n)$ $0 \leq j \leq S$	$O(p(n))$
$B_{i,k}$	$i \rightarrow$ pasul de execuție $k \rightarrow$ locația unde se află capul de bandă	$0 \leq i \leq p(n)$ $0 \leq k \leq p(n)$	$O(p(n)^2)$
$T_{i,l,c}$	$i \rightarrow$ pasul de execuție $l \rightarrow$ locația de pe bandă $c \rightarrow$ caracterul scris la poziția l	$0 \leq i \leq p(n)$ $0 \leq l \leq p(n)$ $1 \leq i \leq \|\Sigma\|$	$O(p(n)^2)$

În continuare, trebuie să proiectăm ϕ astfel încât aceasta să fie satisfiabilă doar dacă există o secvență formată din $p(n)$ tranziții de la configurația inițială până la o configurație care corespunde unei stări de acceptare a intrării. Concret, vom introduce următoarele clauze:

1. Pentru a descrie configurația inițială:
   - $Q_{0,I}$, $B_{0,0}$ (NT se află, la primul pas, în starea inițială I, respectiv capul de bandă se află pe prima poziție)
   - T_{0,k,w_i} $\forall\ 0 \leq k \leq |w|$ (pe primele $|w|$ celule se află datele de intrare)
   - $T_{0,k,BL}$ $\forall\ |w| < k \leq p(n)$ (pe următoarele celule se află caracterul special care semnifică că nicio valoare nu este scrisă)
2. Pentru a descrie configurația finală:
   - $Q_{p(n),q_A}$ (NT se află, la pasul final, într-o stare de acceptare)[4]
3. Pentru fiecare pas i, NT se află într-o unică stare validă:
   - $Q_{i,0} \vee Q_{i,1} \vee \cdots \vee Q_{i,S}$ (cel puțin o stare la fiecare pas)
   - $Q_{i,j_1} \rightarrow \overline{Q_{i,j_2}}$ $\forall j_1, j_2,\ j1 \neq j2,\ 0 \leq j_1, j_2 \leq S$ (cel mult o stare la fiecare pas)
4. Pentru fiecare pas i, B se află într-o unică stare validă:
   - $B_{i,0} \vee B_{i,1} \vee \cdots \vee B_{i,p(n)}$ (cel puțin o poziție pentru capul de bandă)
   - $B_{i,k_1} \rightarrow \overline{B_{i,k_2}}$ $\forall k_1, k_2,\ k1 \neq k2,\ 0 \leq k_1, k_2 \leq p(n)$ (cel mult o poziție pentru capul de bandă)
5. Pentru fiecare pas i, fiecare celulă de pe bandă l are asociată o unică valoare:
   - $T_{i,l,0} \vee T_{i,l,1} \vee \cdots \vee T_{i,l,|\Sigma|}$ (cel puțin o valoare asociată fiecărei celule)
   - $T_{i,l,c_1} \rightarrow T_{i,l,c_2}$ $\forall c_1, c_2, c_1 \neq c_2$ (cel mult o valoare asociată fiecărei celule)
6. Trecerea de la configurația de la pasul i la cea de la pasul $i+1$ respectă specificația funcției de tranziție δ a lui NT:
   - $(Q_{i,j} \wedge H_{i,k} \wedge T_{i,k,c}) \rightarrow \vee_{(j',c',d)\in\delta(j,c)}(Q_{i+1,j'} \wedge B_{i+1,j+d}, T_{i+1,k,c'})$ (unde $d = +1$ corespunde direcției $Right$, respectiv $d = -1$ corespunde direcției $Left$)
7. La fiecare pas, celulele rămân neschimbate dacă nu acționează asupra lor capul de bandă.
   - $T_{i,l,c} \wedge T_{i+1,l,c'} \rightarrow B_{i,l}, \forall c \neq c'$

Fiecare dintre clauzele menționate pot fi rescrise în forma CNF pentru a compune formula ϕ. Contribuția fiecăreia la dimensiunea formulei este suprinsă în tabelul următor:

[4] Convențional, considerăm că, o dată ajunsă în starea q_A, configurația NT rămâne neschimbată la următorii pași.

Descriere clauză	Dimensiune (număr variabile)
Configurația inițială	$O(p(n))$
Configurația finală	$O(1)$
O unică stare validă la fiecare pas	$O(p(n))$
O unică valoare asociată fiecărei celule	$O(p(n)^3)$
Respectarea funcției de tranziție	$O(p(n)^2)$
Celulele rămân neschimbate dacă nu acționează asupra lor capul de bandă	$O(p(n)^2)$

Deoarece indecșii unei variabile ajung până la $p(n)$, pentru a include o variabilă într-o clauză sunt necesari $O(log(p(n)))$ biți. Prin urmare, dimensiunea totală a formulei obținute este $O(p(n)^3 log(p(n))$, adică polinomială în raport cu dimensiunea intrării.

Pentru a finaliza reducerea vom constata că, dacă NT acceptă intrarea w, atunci ϕ este satisfiabilă prin asignarea valorilor de adevăr corespunzătoare fiecărei variabile. De asemenea, dacă ϕ este satisfiabilă, putem reproduce o secvența de configurații din starea inițială până la o stare de acceptare pentru a valida că $NT(w) = 1$. □

Consecință: Deoarece relația $<_p$ este tranzitivă, pentru a demonstra că o altă problemă $Prob$ este NPC este suficient să arătăm că $Prob \in NP$ și SAT sau altă problemă din NPC se reduce polinomial la $Prob$.

5.3 $3SAT \in NP - Complete$

k-SAT este varianta problemei SAT în care fiecare clauză are exact k literali. Pentru a arăta că 3-SAT este NP-Complete vom face o reducere de la problema mai generală SAT ($SAT \leq_p 3 - SAT$).

Demonstrație. **Plan demonstrație:** Ne propunem să modificăm separat fiecare clauză din formula inițială (ϕ), înlocuind-o cu una sau mai multe clauze cu 3 literali, astfel încât noua formulă rezultată (ϕ') să fie echivalentă semantic.

Fie C o clauză arbitrară din ϕ. Atunci, dacă:

1. C **este compusă dintr-un singur literal** (x):
   Introducem două variabile noi (z_1 și z_2) și înlocuim C cu următoarele clauze:
   $$(x \vee z_1 \vee z_2), (x \vee \overline{z_1} \vee z_2), (x \vee z_1 \vee \overline{z_2}), (x \vee \overline{z_1} \vee \overline{z_2})$$
2. C **este compusă din doi literali** $(x_1 \vee x_2)$:
   Introducem o variabilă nouă z_1 și înlocuim C cu următoarele clauze:
   $$(x_1 \vee x_2 \vee z_1), (x_1 \vee x_2 \vee \overline{z_1})$$
3. C **este compusă din trei literali**:
   Lăsăm clauza C nemodificată.
4. C **este compusă din** k **literali, cu** $k > 3$ $(x_1 \vee x_2 \vee \cdots \vee x_k)$:
   Introducem $k - 3$ variabile auxiliare $(z_1, z_2, \ldots, z_{k-3})$ și înlocuim C cu următoarele clauze:
   $$(x_1 \vee x_2 \vee z_1), (\overline{z_1} \vee x_3 \vee z_2), (\overline{z_2} \vee x_4 \vee z_3), \ \ldots \ , (\overline{z_{k-3}} \vee x_{k-1} \vee x_k)$$

Observăm că:

- Dacă clauza inițială era satisfiabilă, atunci există o posibilitate de a asigna valori de adevăr variabilelor auxiliare astfel încât noile clauze introduse să fie satisfiabile (pentru cazurile 1 și 2 orice asignare este validă).
- Dacă clauza inițială nu era satisfiabilă, nu există nicio posibilitate de a asigna valori de adevăr variabilelor auxiliare astfel încât noile clauze introduse să fie satisfiabile.

Prin urmare, $SAT(\phi) = True \iff SAT(\phi') = True$. De asemenea, transformarea este polinomială în raport cu dimensiunea intrării. □

References

[1] Thomas H Cormen et al. *Introduction to algorithms*. MIT press, 2022.

[2] Michael R Garey and David S Johnson. *Computers and intractability*. Vol. 174. Freeman San Francisco, 1979.

[3] Tim Roughgarden. *Algorithms illuminated*. Soundlikeyourself publishing, 2017.

[4] Michael Sipser. *Introduction to the Theory of Computation*. English. 3rd ed. Cengage Learning, 2012. ISBN: 978-1133187790.